
Documentación Técnica

Sistema de Distribución de Recursos en Redes mediante el Método CPM con Interfaz Gráfica en Python

Investigación de Operaciones II — IO2-82

Técnicas de Planeación de Redes
Distribución de Recursos

Grupo: 9

Asignatura: Investigación de Operaciones II

Autores: Juan Esteban Vidal Cabezas - 20241020059

David Mauricio Pardo Jiménez - 20241020181

Santiago Alfonso Arias - 20222020093

Jose Fernando Ramirez Ortiz 20241020080

21 de mayo de 2026

Índice

1. Introducción	3
1.1. Objetivos	3
2. Planteamiento del Problema	3
2.1. Descripción del proyecto	3
2.2. Restricción de recursos	4
2.3. Variables de decisión	4
3. Fundamentos Matemáticos	4
3.1. El Método de la Ruta Crítica (CPM)	4
3.1.1. El Diagrama de Gantt como herramienta de visualización	5
3.1.2. Elementos del diagrama	5
3.1.3. Procedimiento de construcción	5
3.1.4. Lectura del diagrama para el proyecto de construcción	6
3.1.5. Holgura total y ruta crítica	6
3.2. Perfil de recursos	6
3.3. Nivelación de recursos	7
3.3.1. Restricciones	7
3.3.2. Función objetivo	7
3.3.3. Heurística implementada	7
4. Resultados del Modelo CPM	8
4.1. Tabla CPM completa	8
4.2. Ruta crítica y duración mínima	8
4.3. Resumen de nivelación	8
5. Descripción del Código Fuente	8
5.1. Estructura general del sistema	9
5.2. Módulo 1: Datos del proyecto	9
5.3. Módulo 2: Construcción de la red (<code>construir_red</code>)	10
5.4. Módulo 3: Algoritmo CPM (<code>calcular_cpm</code>)	10
5.5. Módulo 4: Perfil y nivelación de recursos	12
5.5.1. Función <code>calcular_perfil_recursos</code>	12
5.5.2. Función <code>nivelar_recursos</code>	12
5.6. Módulo 5: Interfaz gráfica (CPMApp)	13
5.6.1. Inicialización y arquitectura	13
5.6.2. Método central: <code>_calcular</code>	14
5.6.3. Generación de gráficas embebidas	15
5.6.4. Pestañas de la interfaz	16
5.6.5. Concurrencia: hilo para la consulta al agente IA	16
5.7. Módulo 6: Agente de IA local con llamafife	17
5.7.1. ¿Qué es llamafife?	17
5.7.2. Arquitectura del agente	17
5.7.3. Puesta en marcha del servidor	18
5.7.4. Serialización del contexto CPM	18
5.7.5. Comunicación HTTP con la API	19

5.7.6. Construcción de los prompts especializados	21
5.7.7. Tipos de análisis disponibles	22
6. Aplicación a la Distribución de Recursos en Redes	22
6.1. El problema de distribución de recursos	23
6.2. Conexión entre el modelo y la teoría de redes	23
6.3. Impacto de la ruta crítica en la gestión de recursos	23
6.4. Métricas de evaluación de la nivelación	23
6.5. Limitaciones del enfoque heurístico	24
7. Dependencias y Entorno de Ejecución	24
7.1. Requisitos de software	24
7.2. Instalación del entorno virtual	24
8. Conclusiones	25
Referencias	25

1. Introducción

El presente documento describe el diseño, la fundamentación matemática y la implementación computacional de un sistema de planificación de proyectos basado en el **Método de la Ruta Crítica** (*Critical Path Method*, CPM), con soporte para **nivelación de recursos** y una **interfaz gráfica interactiva** desarrollada en Python mediante la biblioteca `tkinter`.

El sistema fue desarrollado en el marco del curso de Investigación de Operaciones (IO2-82) como solución integral al problema de distribución de recursos en redes de proyectos. Adicionalmente, incorpora un módulo experimental de análisis asistido por inteligencia artificial local, a través del servidor `llamafire` (modelo Mistral-7B-Instruct).

1.1. Objetivos

- Modelar y resolver un proyecto de construcción de ocho actividades mediante el algoritmo CPM.
- Identificar la ruta crítica y calcular las holguras de cada actividad.
- Construir y analizar el perfil de recursos diarios, detectando sobrecargas respecto a la disponibilidad máxima.
- Aplicar una heurística de nivelación de recursos para reducir la varianza del perfil sin alterar la duración mínima del proyecto.
- Presentar los resultados en una interfaz gráfica de escritorio con múltiples vistas (tabla CPM, red, Gantt y perfil de recursos).

2. Planteamiento del Problema

2.1. Descripción del proyecto

Se modela la construcción de una **planta industrial** compuesta por ocho actividades interdependientes. Cada actividad posee una duración fija (en días), una demanda de trabajadores (recursos) y un conjunto de predecesoras que deben completarse antes de que pueda iniciarse.

Cuadro 1: Datos del proyecto: actividades, duraciones, recursos y precedencias

ID	Actividad	Duración (d)	Recursos (trab./d)	Predecesoras
A	Excavación	3	4	—
B	Cimentación	5	6	A
C	Estructura metálica	4	8	B
D	Inst. eléctrica	3	3	B
E	Inst. hidráulica	2	3	B
F	Techado	3	5	C
G	Acabados	4	6	D, E, F
H	Equipamiento	2	4	G

2.2. Restricción de recursos

El proyecto cuenta con un límite de **12 trabajadores disponibles por día**. Cualquier día en que la suma de recursos de las actividades en ejecución supere este límite constituye una *sobrecarga* que debe mitigarse mediante la nivelación.

2.3. Variables de decisión

Para cada actividad $i \in \mathcal{A} = \{A, B, C, D, E, F, G, H\}$ se define:

- d_i : duración de la actividad i .
- r_i : demanda de recursos de la actividad i .
- ES_i : inicio más temprano (*Early Start*).
- EF_i : finalización más temprana (*Early Finish*).
- LS_i : inicio más tardío (*Late Start*).
- LF_i : finalización más tardía (*Late Finish*).
- TF_i : holgura total (*Total Float*).
- s_i : inicio real de la actividad tras la nivelación.

3. Fundamentos Matemáticos

3.1. El Método de la Ruta Crítica (CPM)

Definición: Red de proyecto

Una red de proyecto es un grafo dirigido acíclico (DAG) $G = (\mathcal{A}, E)$, donde cada nodo $i \in \mathcal{A}$ representa una actividad y cada arco $(j, i) \in E$ indica que j es predecesora de i .

3.1.1. El Diagrama de Gantt como herramienta de visualización

Definición: Diagrama de Gantt

El **diagrama de Gantt** es una representación gráfica de barras horizontales que muestra la programación temporal de las actividades de un proyecto sobre un eje de tiempo. Fue propuesto por Henry L. Gantt en 1910 y constituye hoy uno de los instrumentos más utilizados en la gestión de proyectos, especialmente en combinación con el método CPM.

3.1.2. Elementos del diagrama

Un diagrama de Gantt construido a partir de los resultados CPM contiene los siguientes elementos:

1. **Eje horizontal (tiempo):** representa la escala de días del proyecto, desde $t = 0$ hasta T^* . Cada unidad corresponde a un día de trabajo.
2. **Eje vertical (actividades):** lista todas las actividades del proyecto, generalmente ordenadas por su inicio más temprano ES_i o por orden topológico en la red.
3. **Barra de ejecución:** para cada actividad i , se dibuja una barra sólida desde ES_i hasta $EF_i = ES_i + d_i$. Su longitud es proporcional a la duración d_i .
4. **Barra de holgura:** para actividades no críticas ($TF_i > 0$), se extiende una barra semitransparente o discontinua desde EF_i hasta LF_i , representando el margen de desplazamiento disponible sin afectar T^* .
5. **Indicador de nivelación:** un marcador especial ($\blacktriangledown$ en la implementación gráfica) señala el inicio real s_i de las actividades que fueron desplazadas durante la nivelación de recursos, cuando $s_i \neq ES_i$.
6. **Codificación de color:** las actividades de la ruta crítica ($TF_i = 0$) se distinguen visualmente de las no críticas mediante colores diferenciados, facilitando la identificación inmediata de las actividades que condicionan la duración total del proyecto.

3.1.3. Procedimiento de construcción

El diagrama de Gantt se construye a partir de la tabla CPM en tres pasos:

Algoritmo 3.1 (Construcción del diagrama de Gantt). 1. **Calcular los tiempos**

CPM: obtener ES_i , EF_i , LS_i , LF_i y TF_i para cada actividad mediante el algoritmo CPM (paso directo e inverso sobre el DAG).

2. **Trazar las barras de ejecución:** para cada actividad i , dibujar una barra horizontal en el intervalo $[ES_i, EF_i]$. Las actividades críticas ($TF_i = 0$) se resaltan con un color distinto.
3. **Agregar las barras de holgura:** para cada actividad con $TF_i > 0$, extender la barra de forma discontinua en el intervalo $[EF_i, LF_i]$, de longitud TF_i días. Esto

permite visualizar cuánto puede retrasarse cada actividad sin comprometer la fecha de entrega del proyecto.

3.1.4. Lectura del diagrama para el proyecto de construcción

Para el proyecto modelado (Tabla 1), el diagrama de Gantt revela las siguientes relaciones temporales:

- Las actividades **A, B, C, F, G, H** forman la ruta crítica y sus barras aparecen en secuencia continua sin margen de holgura. Cualquier retraso en alguna de ellas desplaza la fecha de entrega.
- La actividad **D** (Inst. eléctrica) inicia en $ES_D = 8$, pero puede comenzar hasta el día 10 ($LS_D = 10$) sin afectar el proyecto. Su barra de holgura se extiende 2 días a la derecha de su finalización.
- La actividad **E** (Inst. hidráulica) posee la mayor flexibilidad con $TF_E = 3$ días: puede iniciarse entre los días 8 y 11, lo que la convierte en la principal candidata para absorber variaciones en la demanda de recursos.
- La nivelación de recursos, representada por los marcadores ▼, muestra los desplazamientos aplicados a D y E para reducir la varianza del perfil de recursos, manteniendo en todo momento los inicios dentro del rango $[ES_i, ES_i + TF_i]$.

Nota

El diagrama de Gantt no reemplaza al análisis CPM: es una herramienta de *comunicación* que traduce los resultados numéricos del CPM a una representación visual intuitiva. La información de holguras y ruta crítica proviene exclusivamente del cálculo CPM subyacente.

3.1.5. Holgura total y ruta crítica

$$TF_i = LS_i - ES_i = LF_i - EF_i \quad (1)$$

Definición: Actividad crítica y ruta crítica

Una actividad i es **crítica** si y solo si $TF_i = 0$. La **ruta crítica** es el camino más largo en el DAG, compuesto exclusivamente por actividades críticas. Cualquier retraso en una actividad crítica retrasa el proyecto completo.

3.2. Perfil de recursos

El **perfil de recursos** en el día t es la suma de trabajadores de todas las actividades en ejecución:

$$R(t) = \sum_{i \in \mathcal{A}} r_i \cdot \mathbf{1}[s_i \leq t < s_i + d_i], \quad t = 0, 1, \dots, T^* - 1 \quad (2)$$

donde s_i es el inicio real de la actividad i (antes de nivelar, $s_i = ES_i$).

3.3. Nivelación de recursos

La **nivelación de recursos** busca suavizar el perfil $R(t)$ sin extender T^* , desplazando actividades no críticas dentro de su holgura disponible.

3.3.1. Restricciones

Para toda actividad no crítica i con $TF_i > 0$:

$$ES_i \leq s_i \leq ES_i + TF_i \quad (3)$$

Las actividades críticas permanecen fijas: $s_i = ES_i$ si $TF_i = 0$.

3.3.2. Función objetivo

Se minimiza la varianza del perfil de recursos:

$$\min_{s_i} \text{Var}[R(t)] = \frac{1}{T^*} \sum_{t=0}^{T^*-1} (R(t) - \bar{R})^2 \quad (4)$$

donde $\bar{R} = \frac{1}{T^*} \sum_{t=0}^{T^*-1} R(t)$ es el consumo medio diario.

3.3.3. Heurística implementada

Se utiliza una heurística de búsqueda local greedy:

Algoritmo 3.2 (Nivelación greedy por varianza). 1. Ordenar las actividades no críticas por holgura descendente.

2. Para cada actividad i en ese orden:

a) Calcular la varianza actual del perfil.

b) Para cada desplazamiento $\delta \in \{1, 2, \dots, TF_i\}$: fijar $s_i = ES_i + \delta$ y calcular la nueva varianza.

c) Asignar el δ que produce la menor varianza.

3. Devolver el vector de inicios $\{s_i\}$.

4. Resultados del Modelo CPM

4.1. Tabla CPM completa

Cuadro 2: Resultados del algoritmo CPM para el proyecto de construcción

ID	Nombre	d_i	r_i	ES	EF	LS	LF	TF	Crítica
A	Excavación	3	4	0	3	0	3	0	Sí
B	Cimentación	5	6	3	8	3	8	0	Sí
C	Estr. metálica	4	8	8	12	8	12	0	Sí
D	Inst. eléctrica	3	3	8	11	10	13	2	No
E	Inst. hidráulica	2	3	8	10	11	13	3	No
F	Techado	3	5	12	15	12	15	0	Sí
G	Acabados	4	6	15	19	15	19	0	Sí
H	Equipamiento	2	4	19	21	19	21	0	Sí

4.2. Ruta crítica y duración mínima

Aplicando las ecuaciones (??)–(??):

$$T^* = 21 \text{ días}$$

$$\text{Ruta crítica: } A \rightarrow B \rightarrow C \rightarrow F \rightarrow G \rightarrow H$$

Las actividades con holgura disponible son:

- **D** (Inst. eléctrica): $TF_D = 2$ días — puede desplazarse hasta 2 días.
- **E** (Inst. hidráulica): $TF_E = 3$ días — puede desplazarse hasta 3 días.

4.3. Resumen de nivelación

Cuadro 3: Comparación del perfil de recursos antes y después de la nivelación

Métrica	Sin nivelar	Nivelado
Varianza del perfil	—	Reducida
Pico de recursos	>12 posible	≤ 12
Días con sobrecarga	Varios	Minimizados

5. Descripción del Código Fuente

5.1. Estructura general del sistema

El sistema está implementado en un único archivo Python (`cpm_gui.py`) organizado en cinco capas funcionales bien diferenciadas. Esta separación permite modificar cualquier capa sin afectar las demás: por ejemplo, cambiar la heurística de nivelación sin tocar la interfaz, o reemplazar `tkinter` por otra biblioteca gráfica sin alterar el núcleo CPM.

Cuadro 4: Capas funcionales del sistema y su responsabilidad

Capa	Nombre	Responsabilidad
1	Datos	Diccionario <code>ACTIVIDADES_DEFAULT</code> y constante <code>RECURSOS_DISP_DEFAULT</code> . Centraliza todos los parámetros del proyecto.
2	Lógica CPM	Funciones puras <code>construir_red</code> , <code>calcular_cpm</code> , <code>obtener_ruta_critica</code> , <code>calcular_perfil_recursos</code> y <code>nivelar_recursos</code> . Sin estado de interfaz.
3	Visualización	Gráficas de red, Gantt e histogramas generadas con <code>matplotlib</code> y embebidas en la ventana mediante <code>FigureCanvasTkAgg</code> .
4	Interfaz	Clase <code>CPMApp</code> que hereda de <code>tk.Tk</code> . Gestiona el ciclo de vida de la ventana, las pestañas y la actualización de vistas.
5	Agente IA	Funciones <code>_llamar_ia</code> y <code>construir_contexto_cpm</code> . Comunicación HTTP con el servidor <code>llamafile</code> local.

5.2. Módulo 1: Datos del proyecto

Los datos del proyecto se almacenan en un diccionario de Python anidado denominado `ACTIVIDADES_DEFAULT`. Esta estructura centraliza toda la información necesaria para el cálculo CPM en un único lugar, facilitando la modificación del proyecto sin alterar el código de cálculo:

Listing 1: Estructura de datos del proyecto

```

1 ACTIVIDADES_DEFAULT = {
2     "A": {"nombre": "Excavacion",      "duracion": 3, "recursos"
3         : 4,
4         "predecesoras": []},
5     "B": {"nombre": "Cimentacion",     "duracion": 5, "recursos"
6         : 6,
7         "predecesoras": ["A"]},
8     "C": {"nombre": "Estr. metalica", "duracion": 4, "recursos"
9         : 8,
10        "predecesoras": ["B"]},

```

```

8     ...
9 }
10 RECURSOS_DISP_DEFAULT = 12  # trabajadores maximos por dia

```

Cada clave del diccionario es el identificador de la actividad (".A", "B", etc.) y su valor es otro diccionario con cuatro campos: **nombre** (etiqueta descriptiva), **duracion** (días), **recursos** (trabajadores requeridos por día) y **predecesoras** (lista de IDs de actividades que deben terminar antes). La lista vacía [] indica que la actividad no tiene predecesoras y puede iniciarse en el día 0.

Al arrancar, la clase `CPMApp` copia este diccionario en `self.actividades`, de modo que futuras versiones del sistema podrían permitir al usuario editar los datos desde la interfaz sin modificar el archivo.

5.3. Módulo 2: Construcción de la red (`construir_red`)

Listing 2: Función `construir_red`: creación del DAG del proyecto

```

1 def construir_red(actividades):
2     G = nx.DiGraph()                                # grafo dirigido
3     vacio
4     for act_id, datos in actividades.items():
5         G.add_node(act_id, **datos)                 # nodo con
6         atributos
7     for act_id, datos in actividades.items():
8         for pred in datos["predecesoras"]:
9             G.add_edge(pred, act_id)                # arco pred ->
10            act_id
11     return G

```

La función recorre el diccionario dos veces deliberadamente: primero agrega todos los nodos (para que existan antes de conectarlos), y luego agrega los arcos de precedencia. `nx.DiGraph()` crea un grafo dirigido vacío de la biblioteca `networkx`. El operador `**datos` desempaqueta el diccionario de atributos de cada actividad y los asocia al nodo como metadatos accesibles posteriormente con `G.nodes[act_id]["duracion"]`, etc.

Cada arco (`pred, act_id`) representa la relación “*pred* debe finalizar antes de que *act_id* pueda comenzar”. El resultado es un DAG con 8 nodos y tantos arcos como relaciones de precedencia existan en el proyecto.

5.4. Módulo 3: Algoritmo CPM (`calcular_cpm`)

Listing 3: Función `calcular_cpm`: implementación completa del CPM

```

1 def calcular_cpm(G, actividades):
2     # Orden topologico: garantiza que cada nodo se procesa
3     # despues de TODOS sus predecesores
4     orden = list(nx.topological_sort(G))
5

```

```

6      # PASO DIRECTO

7      ES, EF = {}, {}
8      for act in orden:
9          preds = list(G.predecessors(act))
10         # Si no hay predecesoras, ES = 0 (inicio del proyecto)
11         ES[act] = max((EF[p] for p in preds), default=0)
12         EF[act] = ES[act] + actividades[act]["duracion"]
13     T_star = max(EF.values()) # duracion minima del proyecto
14
15     # PASO INVERSO

16     LF, LS = {}, {}
17     for act in reversed(orden):
18         succs = list(G.successors(act))
19         # Si no hay sucesoras, LF = T_star (fin del proyecto)
20         LF[act] = min((LS[s] for s in succs), default=T_star)
21         LS[act] = LF[act] - actividades[act]["duracion"]
22
23     # HOLGURAS

24     TF = {act: LS[act] - ES[act] for act in orden}
25
26     # CONSTRUCCION DEL DATAFRAME

27     resultados = []
28     for act in orden:
29         resultados.append({
30             "Actividad": act,
31             "Nombre": actividades[act]["nombre"],
32             "Duracion": actividades[act]["duracion"],
33             "Recursos": actividades[act]["recursos"],
34             "ES": ES[act], "EF": EF[act],
35             "LS": LS[act], "LF": LF[act],
36             "Holgura (TF)": TF[act],
37             "Critica": TF[act] == 0,
38         })
39     return pd.DataFrame(resultados), T_star, ES, EF, LS, LF,
        TF

```

La función devuelve siete valores: el `DataFrame` con la tabla CPM completa, la duración mínima `T_star` y los seis diccionarios de tiempos (`ES`, `EF`, `LS`, `LF`, `TF`). Estos diccionarios son los que se pasan al resto del sistema: la función de nivelación los usa para calcular desplazamientos válidos, las gráficas los usan para posicionar barras y nodos, y el módulo de IA los serializa en el contexto textual.

El uso de `nx.topological_sort(G)` delega en `networkx` la tarea de ordenar los nodos

respetando todas las precedencias. Si el grafo contuviera un ciclo (error en los datos de entrada), esta función lanzaría una excepción `NetworkXUnfeasible` antes de realizar cualquier cálculo.

5.5. Módulo 4: Perfil y nivelación de recursos

5.5.1. Función `calcular_perfil_recursos`

Listing 4: Cálculo del perfil diario de trabajadores

```

1 def calcular_perfil_recursos(actividades, ES, EF, T_star,
2                             inicio_override=None):
3     perfil = defaultdict(int) # dia -> trabajadores (0 por
4                               defecto)
5     inicio = inicio_override or ES # usa ES si no se indica
6                                     otro
7
8     for act_id, datos in actividades.items():
9         s = inicio[act_id] # dia de inicio de esta
10                               actividad
11         for t in range(s, s + datos["duracion"]):
12             perfil[t] += datos["recursos"] # acumula
13                                             trabajadores
14
15     # Convierte a lista de longitud T_star (dias 0..T_star-1)
16     return [perfil.get(t, 0) for t in range(T_star)]

```

El parámetro `inicio_override` es la clave de diseño de esta función. Cuando se omite, la función calcula el perfil con los inicios tempranos ($s_i = ES_i$), produciendo el histograma original. Cuando se pasa el diccionario `inicio_niv` generado por la nivelación, produce el histograma nivelado. Esta dualidad permite usar la misma función para evaluar cualquier escenario de programación, incluyendo los candidatos intermedios que se prueban durante la búsqueda greedy.

`defaultdict(int)` es un diccionario que devuelve 0 automáticamente para claves inexistentes, evitando verificaciones `if t in perfil` dentro del bucle interno.

5.5.2. Función `nivelar_recursos`

Listing 5: Heurística greedy de nivelación por varianza mínima

```

1 def nivelar_recursos(actividades, ES, EF, LS, LF, TF, T_star):
2     inicio_actual = dict(ES) # copia del estado inicial
3
4     # Ordenar por holgura descendente: mas flexibles primero
5     con_holgura = sorted(
6         [(a, TF[a]) for a in actividades if TF[a] > 0],
7         key=lambda x: -x[1]
8     )
9

```

```

10     for act_id, holgura in con_holgura:
11         # Varianza con el inicio actual (referencia)
12         mejor_varianza = np.var(
13             calcular_perfil_recursos(actividades, ES, EF,
14                                     T_star, inicio_actual))
15         mejor_inicio = inicio_actual[act_id]
16
17         # Probar cada desplazamiento posible (1 a TF_i días)
18         for delta in range(1, holgura + 1):
19             candidato = dict(inicio_actual)           # copia para
20                                                         no mutar
21             candidato[act_id] = ES[act_id] + delta
22             v = np.var(calcular_perfil_recursos(
23                 actividades, ES, EF, T_star, candidato))
24             if v < mejor_varianza:
25                 mejor_varianza = v
26                 mejor_inicio = ES[act_id] + delta
27
28             inicio_actual[act_id] = mejor_inicio      # fijar el mejor
29                                                         delta
30     return inicio_actual

```

La función explora, para cada actividad no crítica, todos los posibles desplazamientos dentro de su holgura y selecciona el que produce la menor varianza en el perfil de recursos. El orden de procesamiento (mayor holgura primero) es una decisión heurística: las actividades con más margen tienen más opciones y su posicionamiento puede condicionar fuertemente el perfil antes de que las actividades menos flexibles sean evaluadas.

La instrucción `candidato = dict(inicio_actual)` crea una copia superficial del diccionario de inicios. Esto es necesario porque `calcular_perfil_recursos` modifica el perfil acumulado internamente; si se pasara el diccionario original, cada iteración del bucle afectaría los resultados de las siguientes.

5.6. Módulo 5: Interfaz gráfica (CPMApp)

5.6.1. Inicialización y arquitectura

La clase `CPMApp` hereda directamente de `tk.Tk`, lo que la convierte en la ventana principal de la aplicación. Al instanciarse, ejecuta `__init__` que configura la ventana, inicializa todos los atributos de estado en `None` y llama a dos métodos en secuencia:

Listing 6: Constructor de `CPMApp`

```

1 class CPMApp(tk.Tk):
2     def __init__(self):
3         super().__init__()
4         self.title("CPM      Distribucion de Recursos | IO2
5                     -82")
6         self.geometry("1280x800")
7         self.configure(bg=BG_DARK)

```

```

8         # Copia de trabajo de los datos del proyecto
9         self.actividades = {k: dict(v) for k, v in
10                                ACTIVIDADES_DEFAULT.items()}
11         self.recursos_disp = RECURSOS_DISP_DEFAULT
12
13         # Resultados CPM (None hasta que se calcule)
14         self.df = self.T_star = self.ES = self.EF = None
15         self.LS = self.LF = self.TF = self.G = None
16
17         self._build_ui()      # construye todos los widgets
18         self._calcular()      # calcula CPM al arrancar

```

El método `_build_ui()` construye toda la jerarquía de widgets de Tkinter (cabecera, Notebook, seis pestañas, tablas, áreas de texto y botones), pero no realiza ningún cálculo. El método `_calcular()` se llama inmediatamente después para poblar todas las vistas con los datos del proyecto por defecto, de modo que la ventana muestra resultados desde el primer instante.

5.6.2. Método central: `_calcular`

Listing 7: Método `_calcular`: orquestador del sistema

```

1 def _calcular(self):
2     self.recursos_disp = self.var_recursos.get() # leer
3     spinbox
4
5     # 1. Ejecutar toda la cadena de calculo CPM
6     self.G = construir_red(self.actividades)
7     self.df, self.T_star, self.ES, self.EF, \
8     self.LS, self.LF, self.TF = calcular_cpm(self.G,
9                                                self.actividades
10                                                )
11     _, self.rutas = obtener_ruta_critica(self.G, self.TF)
12
13     # 2. Calcular perfiles de recursos
14     self.perfil_orig = calcular_perfil_recursos(
15         self.actividades, self.ES, self.EF, self.T_star)
16     self.inicio_niv = nivelar_recursos(
17         self.actividades, self.ES, self.EF,
18         self.LS, self.LF, self.TF, self.T_star)
19     self.perfil_niv = calcular_perfil_recursos(
20         self.actividades, self.ES, self.EF,
21         self.T_star, self.inicio_niv)
22
23     # 3. Actualizar todas las vistas
24     self._actualizar_resumen()
25     self._actualizar_tabla()
26     self._graficar_red()
27     self._graficar_gantt()
28     self._graficar_perfil()

```

Este método actúa como el controlador central del patrón MVC. Cada vez que el usuario presiona “Calcular CPM” (o al arrancar la aplicación), este método ejecuta en orden: lectura de parámetros desde la UI, toda la cadena de cálculo CPM, y finalmente la actualización de las cinco vistas activas. El bloque `try/except` que lo envuelve captura cualquier error en los datos de entrada (por ejemplo, un ciclo en el grafo) y lo muestra en un cuadro de diálogo sin bloquear la aplicación.

5.6.3. Generación de gráficas embebidas

Las gráficas de matplotlib se integran en la ventana de Tkinter mediante el backend `TkAgg`. El patrón utilizado es idéntico en los tres métodos de graficación:

Listing 8: Patrón de integración matplotlib + Tkinter

```

1 def _graficar_gantt(self):
2     frame = self.canvas_gantt_frame
3     for w in frame.winfo_children():
4         w.destroy()           # eliminar grafica anterior
5
6     fig, ax = plt.subplots(figsize=(12, 5))
7     fig.patch.set_facecolor(BG_DARK) # fondo oscuro
8     # ... dibujar barras, etiquetas, etc. ...
9
10    canvas = FigureCanvasTkAgg(fig, master=frame)
11    canvas.draw()
12    canvas.get_tk_widget().pack(fill="both", expand=True)
13    NavigationToolbar2Tk(canvas, frame).pack(fill="x")
14    plt.close(fig)           # liberar memoria de matplotlib

```

La llamada `for w in frame.winfo_children(): w.destroy()` destruye la gráfica anterior antes de crear la nueva. Esto es necesario porque cada llamada a `_calcular()` regenera todas las vistas; sin este paso, las gráficas se apilarían en memoria consumiendo recursos crecientes. `NavigationToolbar2Tk` añade la barra de herramientas estándar de matplotlib (zoom, pan, guardar imagen) directamente en la ventana de Tkinter. `plt.close(fig)` libera la figura de la memoria de matplotlib una vez que el canvas de Tkinter la ha renderizado.

5.6.4. Pestañas de la interfaz

Cuadro 5: Descripción de las pestañas de la interfaz gráfica

N.º	Pestaña	Contenido y widgets principales
1	Datos del proyecto	<code>ttk.Treeview</code> con las actividades del proyecto. <code>tk.Spinbox</code> para ajustar el límite de recursos. <code>scrolledtext.ScrolledText</code> con el resumen numérico del CPM (duración, ruta crítica, métricas de nivelación).
2	Tabla CPM	<code>ttk.Treeview</code> con las 10 columnas del CPM. Filas rojas = actividades críticas; verdes = con holgura. Tarjetas <code>tk.Frame</code> con métricas globales.
3	Red CPM	<code>FigureCanvasTkAgg</code> con el grafo del proyecto. Nodos rojos = críticos; verdes = con holgura. Barra de navegación matplotlib para zoom y exportación.
4	Gantt	<code>FigureCanvasTkAgg</code> con el diagrama de Gantt. Barras sólidas = duración; semitransparentes = holgura; marcador ▼ = desplazamiento por nivelación.
5	Perfil de Recursos	<code>FigureCanvasTkAgg</code> con dos histogramas comparativos. Línea discontinua = límite; línea punteada = promedio. Barras rojas = días con sobrecarga.
6	Análisis IA	Tres <code>tk.Radiobutton</code> para seleccionar el tipo de análisis. Botón que lanza el hilo de consulta. <code>scrolledtext.ScrolledText</code> con la respuesta del LLM.

5.6.5. Concurrencia: hilo para la consulta al agente IA

Listing 9: Manejo de concurrencia para la consulta al LLM

```

1 def _lanzar_ia(self):
2     self.lbl_ia_estado.configure(text="Consultando IA...")
3     self.txt_ia.configure(state="normal")
4     self.txt_ia.delete("1.0", "end")
5     self.txt_ia.insert("end", "Enviando datos al servidor...")
6     self.txt_ia.configure(state="disabled")
7     # Lanzar la consulta en un hilo separado
8     threading.Thread(target=self._ia_worker, daemon=True).
        start()
9
10 def _ia_worker(self):
11     tipo = self.var_ia_tipo.get()      # tipo de analisis
        elegido

```

```

12     ctx      = construir_contexto_cpm(...)
13     prompt = prompts[tipo][1]           # prompt específico
14     respuesta = _llamar_ia(prompt)       # bloquea hasta recibir
                                         respuesta
15     # Devolver control al hilo principal de Tkinter
16     self.after(0, self._ia_mostrar, titulo, respuesta)
17
18 def _ia_mostrar(self, titulo, respuesta):
19     self.txt_ia.configure(state="normal")
20     self.txt_ia.delete("1.0", "end")
21     self.txt_ia.insert("end", respuesta)
22     self.txt_ia.configure(state="disabled")
23     self.lbl_ia_estado.configure(text="Análisis recibido")

```

Tkinter no es *thread-safe*: solo el hilo principal puede modificar widgets. Por ello, la consulta HTTP se ejecuta en un hilo secundario (`threading.Thread`) mientras que la actualización de la UI se delega al hilo principal a través de `self.after(0, callback)`. El argumento `daemon=True` hace que el hilo secundario se termine automáticamente si el usuario cierra la ventana, evitando procesos huérfanos. El flujo completo es: hilo principal lanza hilo secundario → hilo secundario llama a `_llamar_ia` (bloqueante, puede tardar minutos) → al recibir respuesta, programa `_ia_mostrar` en el hilo principal vía `after(0, ...)` → hilo principal actualiza la UI.

5.7. Módulo 6: Agente de IA local con llamafle

5.7.1. ¿Qué es llamafle?

Definición: llamafle

llamafle es una tecnología desarrollada por Mozilla que empaqueta un modelo de lenguaje de gran escala (LLM) junto con el motor de inferencia `llama.cpp` en un **único archivo ejecutable** autocontenido. Al iniciarse, levanta un servidor HTTP local que expone una API compatible con el estándar OpenAI `/v1/chat/completions`, permitiendo interactuar con el modelo desde cualquier lenguaje de programación mediante peticiones HTTP ordinarias, sin necesidad de conexión a Internet ni servicios en la nube.

El modelo utilizado en este proyecto es **Mistral-7B-Instruct-v0.2** en su variante cuantizada Q2_K, que reduce el tamaño del modelo de ~14 GB (FP16) a ~3 GB mediante cuantización de 2 bits por peso, haciéndolo viable en equipos de escritorio con RAM limitada.

5.7.2. Arquitectura del agente

El flujo completo del agente de IA sigue el siguiente proceso:

1. **Serialización del contexto CPM:** la función `construir_contexto_cpm()` transforma todos los resultados numéricos del modelo (tabla CPM, perfil de recursos,

métricas de nivelación, actividades desplazadas) en un bloque de texto estructurado que el LLM puede leer e interpretar.

2. **Construcción del prompt:** el contexto serializado se concatena con una instrucción de rol (*system prompt implícito*) y las preguntas específicas del análisis solicitado. El prompt completo se inyecta como mensaje de usuario en la conversación.
3. **Inferencia local:** el servidor llamafire recibe el prompt vía HTTP POST, lo procesa con el motor llama.cpp y devuelve la respuesta generada por el modelo Mistral-7B.
4. **Presentación:** la respuesta se muestra en la pestaña “Análisis IA” de la interfaz, formateada con ajuste de línea para facilitar su lectura.

5.7.3. Puesta en marcha del servidor

Antes de ejecutar el sistema, el servidor llamafire debe iniciarse de forma independiente desde la línea de comandos:

Listing 10: Inicio del servidor llamafire en Windows (PowerShell)

```

1 # Renombrar el archivo (solo la primera vez en Windows)
2 Rename-Item .\mistral-7b-instruct-v0.2.Q2_K.llamafire '
3     .\mistral-7b-instruct-v0.2.Q2_K.exe
4
5 # Iniciar el servidor en el puerto 8080
6 .\mistral-7b-instruct-v0.2.Q2_K.exe --server '
7     --host 0.0.0.0 --port 8080

```

Una vez iniciado, el servidor queda escuchando en `http://localhost:8080` y permanece activo hasta que se cierre manualmente. La aplicación CPM puede conectarse en cualquier momento mientras el servidor esté en ejecución.

5.7.4. Serialización del contexto CPM

La función `construir_contexto_cpm` es el puente entre los resultados numéricos del CPM y el modelo de lenguaje. Transforma diccionarios y listas de Python en un bloque de texto estructurado legible por el LLM:

Listing 11: Función `construir_contexto_cpm`: serialización de resultados

```

1 def construir_contexto_cpm(df, rutas, T_star, perfil_orig,
2     perfil_niv, inicio_niv, ES,
3     recursos_disp):
4     ruta_str = " -> ".join(rutas[0]) if rutas else "N/A"
5
6     # Serializar cada actividad con todos sus atributos CPM
7     actos = []
8     for _, row in df.iterrows():
9         tipo = "CRITICA" if row["Critica"] else \

```

```

9         f"holgura={row['Holgura (TF)']}]d"
10    actos.append(
11        f"    {row['Actividad']} ({row['Nombre']}): "
12        f"dur={row['Duracion']}d, recursos={row['Recursos'
13        ']}, "
14        f"ES={row['ES']}, EF={row['EF']} [{tipo}]"
15    )
16
17    # Serializar el perfil día a día con alertas de sobrecarga
18    perfil_str = [
19        f"    Día {t}: sin nivelar={o}, nivelado={n}"
20        + (" <- SOBRECARGA" if o > recursos_disp else "")
21        for t, (o, n) in enumerate(zip(perfil_orig, perfil_niv
22        ))
23    ]
24
25    return (
26        f"PROYECTO CPM - Duracion: {T_star}d - "
27        f"Recursos disp.: {recursos_disp}/día\n"
28        f"Ruta critica: {ruta_str}\n\n"
29        f"ACTIVIDADES:\n" + "\n".join(actos) +
30        "\n\nPERFIL DE RECURSOS:\n" + "\n".join(perfil_str)
31    )

```

El texto resultante incluye explícitamente: la duración del proyecto, la ruta crítica, los tiempos ES/EF y la holgura de cada actividad, el perfil de recursos día a día con marcadores de sobrecarga, y las actividades desplazadas por la nivelación. Esta información completa permite al LLM razonar sobre el proyecto sin necesidad de acceder a ninguna fuente externa.

5.7.5. Comunicación HTTP con la API

El sistema se comunica con llamafle mediante la biblioteca estándar `urllib` de Python, sin dependencias externas adicionales:

Listing 12: Función `_llamar_ia`: petición HTTP completa al servidor

```

1 def _llamar_ia(prompt, max_tokens=800, temperatura=0.7):
2     payload = {
3         "model": "mistral-7b-instruct-v0.2.Q2_K",
4         "messages": [{"role": "user", "content": prompt}],
5         "max_tokens": max_tokens,
6         "temperature": temperatura,
7     }
8     # Serializar el payload a JSON y codificarlo en bytes UTF-8
9     data = json.dumps(payload).encode("utf-8")
10
11    # Construir la petición HTTP POST
12    req = urllib.request.Request(
13        "http://localhost:8080/v1/chat/completions",

```

```

14     data=data,
15     headers={"Content-Type": "application/json"},
16     method="POST",
17 )
18
19 try:
20     with urllib.request.urlopen(req, timeout=600) as resp:
21         resultado = json.loads(resp.read().decode("utf-8"))
22         # Extraer el texto de la respuesta del LLM
23         return resultado["choices"][0]["message"]["content"]
24         .strip()
25 except urllib.error.URLError as e:
26     return (
27         "[!] No se pudo conectar con llamafile.\n"
28         "Asegurate de que el servidor este corriendo:\n"
29         ".\\mistral-7b-instruct-v0.2.Q2_K.exe "
30         "--server --host 0.0.0.0 --port 8080\n"
31         f"Error: {e}"
32     )
33 except (KeyError, json.JSONDecodeError) as e:
34     return f"[!] Error al interpretar la respuesta: {e}"

```

Los parámetros clave de la petición son:

Cuadro 6: Parámetros de la petición al servidor llamafile

Parámetro	Valor	Descripción
model	mistral-7b-instruct-v0.2.Q2_K	Identificador informativo. llamafile lo ignora e infiere siempre con el modelo cargado al arranque.
messages	Lista de mensajes	Historial en formato OpenAI. Se envía un único mensaje de usuario con el prompt completo (contexto + preguntas).
max_tokens	800	Máximo de tokens generados en la respuesta. Limita el largo de la salida para evitar respuestas excesivamente extensas.
temperature	0.7	Controla la aleatoriedad. Valores cercanos a 0 producen respuestas más deterministas; cercanos a 1, más creativas.
timeout	600s	Tiempo máximo de espera HTTP. Se fija en 10 minutos para tolerar inferencias lentas en hardware limitado.

La respuesta del servidor tiene el formato estándar de la API de OpenAI. La ruta `resultado["choices"][0]["message"]["content"]` extrae el texto generado por el

modelo del JSON de respuesta. Si el servidor no está disponible (`URLError`) o la respuesta no tiene el formato esperado (`KeyError` / `JSONDecodeError`), la función devuelve un mensaje de error descriptivo que se muestra en la pestaña de IA sin interrumpir el resto del sistema.

5.7.6. Construcción de los prompts especializados

Dentro de `_ia_worker`, cada tipo de análisis genera un prompt diferente concatenando el contexto serializado con instrucciones de rol y preguntas específicas:

Listing 13: Construcción dinámica de prompts por tipo de análisis

```

1 def _ia_worker(self):
2     tipo = self.var_ia_tipo.get()      # leer seleccion del
        usuario
3     ctx = construir_contexto_cpm(      # serializar resultados
        CPM
4         self.df, self.rutas, self.T_star,
5         self.perfil_orig, self.perfil_niv,
6         self.inicio_niv, self.ES, self.recursos_disp
7     )
8
9     prompts = {
10         "analisis_cpm": (
11             "ANALISIS DE RUTA CRITICA Y TIEMPOS",
12             ctx + "\n\nEres experto en IO. Responde en espanol
                :\n"
13             "1. Cuales son las actividades mas vulnerables y
                por que?\n"
14             "2. Que implica la holgura de cada actividad no
                critica?\n"
15             "3. Existe riesgo de retraso dado el perfil de
                recursos?"
16         ),
17         "evaluacion_nivelacion": (
18             "EVALUACION DE LA NIVELACION DE RECURSOS",
19             ctx + "\n\nEres especialista en planificacion. "
20             "Responde en espanol:\n"
21             "1. Fue efectiva la nivelacion? Sustenta con
                metricas.\n"
22             "2. Siguen existiendo dias problematicos?\n"
23             "3. Las actividades desplazadas son buenas
                candidatas?"
24         ),
25         "recomendaciones": (
26             "RECOMENDACIONES Y GESTION DE RIESGOS",
27             ctx + "\n\nEres consultor senior de proyectos. "
28             "Responde en espanol:\n"
29             "1. Que tres acciones concretas mejorarian la
                ejecucion?\n"

```

```

30         "2. Como mitigarias los riesgos de la ruta critica
31             ?\n"
32         "3. Si el cliente exige reducir 2 dias, que
33             estrategia aplicarias?"
34     ),
35     }
36     titulo, prompt = prompts[tipo]
37     respuesta = _llamar_ia(prompt)
38     self.after(0, self._ia_mostrar, titulo, respuesta)

```

Cada prompt asigna al modelo un **rol explícito** (experto en IO, especialista en planificación, consultor senior) antes de formular las preguntas. Esta técnica de *role prompting* orienta al modelo a responder desde una perspectiva técnica específica y en el idioma correcto (español), mejorando la calidad y relevancia de las respuestas. El contexto CPM completo se incluye en el mismo mensaje de usuario, ya que llamafile no mantiene estado entre peticiones y cada llamada es independiente.

5.7.7. Tipos de análisis disponibles

1. **Análisis de ruta crítica y tiempos:** el modelo actúa como experto en Investigación de Operaciones e identifica las actividades más vulnerables, interpreta las holguras disponibles y evalúa el riesgo de retraso asociado a la demanda de recursos.
2. **Evaluación de la nivelación de recursos:** el modelo actúa como especialista en planificación de construcción y analiza cuantitativamente la efectividad de la nivelación (varianza, pico, días de sobrecarga), identifica días problemáticos residuales y evalúa si las actividades desplazadas son las candidatas óptimas.
3. **Recomendaciones y gestión de riesgos:** el modelo actúa como consultor senior de proyectos industriales y proporciona tres acciones concretas de mejora, estrategias de mitigación para la ruta crítica y una propuesta de aceleración del proyecto (*crashing* o *fast-tracking*) ante una exigencia de reducción de plazo.

Nota

El agente de IA es un componente **opcional** del sistema. Toda la lógica CPM, la nivelación de recursos y las visualizaciones gráficas funcionan de forma completamente independiente al servidor llamafile. La IA añade una capa de interpretación cualitativa sobre los resultados cuantitativos, pero no interviene en ningún cálculo del modelo.

6. Aplicación a la Distribución de Recursos en Redes

6.1. El problema de distribución de recursos

En la planificación de proyectos mediante redes, la **distribución de recursos** aborda la asignación temporal de recursos limitados (trabajadores, maquinaria, capital) a actividades que compiten por ellos. El problema central es que el schedule más temprano posible (todos los $s_i = ES_i$) puede generar picos de demanda que superen la disponibilidad real, mientras que usar exclusivamente los inicios tardíos ($s_i = LS_i$) traslada los picos al final del proyecto. La solución óptima explota la *holgura disponible* de las actividades no críticas para redistribuir la carga temporal.

6.2. Conexión entre el modelo y la teoría de redes

Relación CPM–Distribución de recursos

El algoritmo CPM proporciona, para cada actividad no crítica, una ventana de flexibilidad $[ES_i, ES_i + TF_i]$ dentro de la cual puede iniciarse sin extender T^* . La nivelación de recursos aprovecha esta flexibilidad para minimizar la varianza del perfil $R(t)$, reduciendo la necesidad de contratar y despedir trabajadores en ciclos cortos.

6.3. Impacto de la ruta crítica en la gestión de recursos

La ruta crítica $A \rightarrow B \rightarrow C \rightarrow F \rightarrow G \rightarrow H$ impone restricciones absolutas: ninguna de sus actividades puede desplazarse. En consecuencia, la estructura de la red determina qué días tienen recursos forzosamente altos (por ejemplo, el día en que se ejecutan C y D/E simultáneamente puede generar una demanda de $8 + 3 = 11$ ó $8 + 3 + 3 = 14$ trabajadores, dependiendo de si D y E se inician en su ES o se retrasan).

La nivelación identifica que:

- **E** (holgura = 3 días) es la candidata prioritaria a desplazarse, ya que su mayor flexibilidad permite más opciones.
- **D** (holgura = 2 días) se evalúa en segundo lugar, condicionada al inicio ya fijado de E.

6.4. Métricas de evaluación de la nivelación

Se emplean tres indicadores cuantitativos para evaluar la calidad de la nivelación:

1. **Varianza del perfil** (ecuación (4)): mide la dispersión del consumo diario. Una varianza menor implica una fuerza de trabajo más estable.
2. **Pico de recursos**: $\max_t R(t)$. Indica la demanda máxima puntual y determina si existe sobrecarga respecto al límite disponible.
3. **Días con sobrecarga**: $|\{t : R(t) > R_{\max}\}|$, donde $R_{\max} = 12$ trabajadores/día. Un valor de cero tras la nivelación significa que el perfil es factible.

6.5. Limitaciones del enfoque heurístico

La heurística implementada es *greedy* y no garantiza la solución óptima global. Para proyectos grandes o con restricciones de recursos más complejas, se recomienda:

- Formulación como **Problema de Programación Entera** (MIP) con variables binarias de precedencia.
- Algoritmos metaheurísticos (*algoritmos genéticos*, recocido simulado).
- Heurísticas clásicas de referencia: Burgess–Killebrew, Wiest–Levy, Método Paralelo con prioridades.

7. Dependencias y Entorno de Ejecución

7.1. Requisitos de software

Cuadro 7: Bibliotecas requeridas por el sistema

Biblioteca	Versión mín.	Uso
networkx	3.0	Construcción y análisis del DAG; orden topológico; rutas.
numpy	1.24	Cálculo de varianza del perfil de recursos.
pandas	2.0	Almacenamiento tabular de resultados CPM.
matplotlib	3.7	Gráficas de red, Gantt e histogramas de recursos.
tkinter	— (stdlib)	Interfaz gráfica de escritorio.

7.2. Instalación del entorno virtual

Listing 14: Creación del entorno virtual en Windows (PowerShell)

```

1 # Crear entorno virtual
2 python -m venv venv_cpm
3
4 # Habilitar ejecucion de scripts (una sola vez)
5 Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope
   CurrentUser
6
7 # Activar el entorno
8 .\venv_cpm\Scripts\Activate.ps1
9
10 # Instalar dependencias
11 pip install networkx numpy pandas matplotlib
12
13 # Ejecutar la aplicacion

```

```
14 python cpm_gui.py
```

8. Conclusiones

1. El algoritmo CPM resuelve de forma exacta y eficiente el problema de programación temporal del proyecto, identificando la ruta crítica $A \rightarrow B \rightarrow C \rightarrow F \rightarrow G \rightarrow H$ con una duración mínima de $T^* = 21$ días.
2. La heurística greedy de nivelación de recursos, basada en minimizar la varianza del perfil diario, aprovecha la holgura de las actividades D (2 días) y E (3 días) para reducir los picos de demanda sin modificar la duración del proyecto.
3. La interfaz gráfica en `tkinter` centraliza en seis pestañas toda la información relevante del modelo: datos, tabla CPM, red, Gantt, perfil de recursos y análisis de IA, facilitando la interpretación de los resultados sin necesidad de herramientas externas.
4. La integración con el modelo de lenguaje local Mistral-7B-Instruct extiende el sistema con capacidades de análisis cualitativo (vulnerabilidades, evaluación de la nivelación, recomendaciones de riesgo), añadiendo valor interpretativo a los resultados cuantitativos del CPM.
5. El diseño modular del código separa claramente la lógica matemática de la presentación, lo que facilita la extensión del sistema (por ejemplo, incorporar otras heurísticas de nivelación o exportar resultados a Excel).

Referencias

- [1] Hillier, F. S., & Lieberman, G. J. (2015). *Introduction to Operations Research* (10.^a ed.). McGraw-Hill Education.
- [2] Taha, H. A. (2017). *Operations Research: An Introduction* (10.^a ed.). Pearson Education.
- [3] Moder, J. J., Phillips, C. R., & Davis, E. W. (1983). *Project Management with CPM, PERT and Precedence Diagramming* (3.^a ed.). Van Nostrand Reinhold.
- [4] Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En *Proceedings of the 7th Python in Science Conference* (SciPy 2008), pp. 11–15.
- [5] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>